

บทที่ 5

SOAP

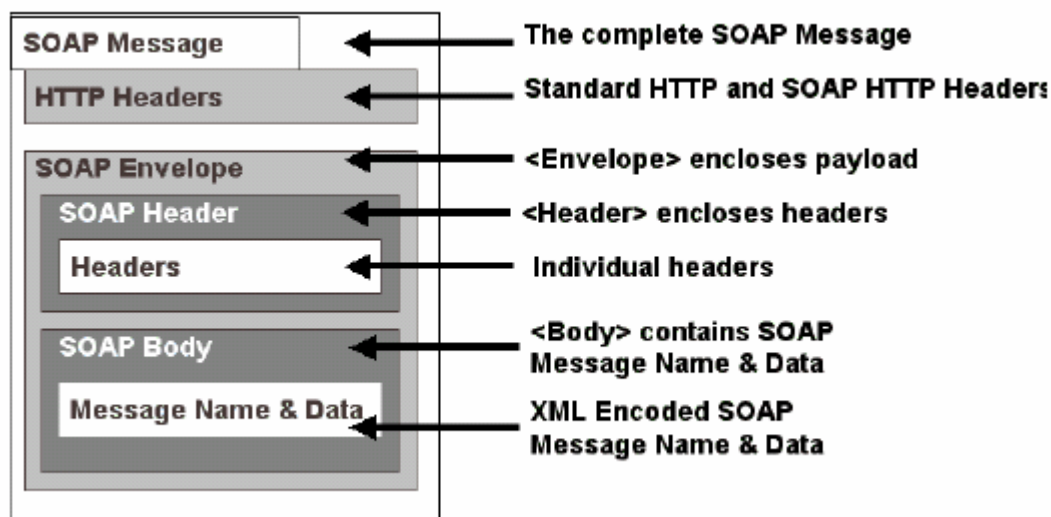
5.1 แนะนำ SOAP

SOAP (Simple Object Access Protocol) เป็นโพรโตคอลที่ใช้ XML เป็นพื้นฐาน เพื่อให้ซอฟต์แวร์คอมพิวเตอร์และแอปพลิเคชันสามารถติดต่อกันผ่าน HTTP ซึ่งเป็นมาตรฐานอินเทอร์เน็ต โพรโตคอลได้เนื่องจากการสื่อสารระหว่างแอปพลิเคชัน เป็นสิ่งจำเป็นต่อการพัฒนาแอปพลิเคชันบนอินเทอร์เน็ตเป็นอย่างมาก แต่แอปพลิเคชันแบบกระจายกันทำงาน (distributed application) ในปัจจุบันใช้ Remote Procedure Call (RPC) สื่อสารกันระหว่างออบเจกต์ เช่น DCOM และ CORBA ซึ่งไม่ได้ใช้พอร์ตเดียวกับ HTTP ที่เป็นพอร์ตมาตรฐานสำหรับให้บริการเว็บ ดังนั้น RPC จึงนำมาปรับใช้กับอินเทอร์เน็ตได้ยาก และมีปัญหาทางด้านความปลอดภัย ไฟร์วอลล์และพร็อกซีเซิร์ฟเวอร์จะไม่ยอมให้ส่งข้อมูลชนิดนี้ได้ตามปกติ วิธีที่ดีกว่าคือใช้ HTTP เพราะเป็นที่ยอมรับโดยอินเทอร์เน็ตเบราว์เซอร์ และเซิร์ฟเวอร์ทุกชนิด ซึ่ง SOAP ถูกสร้างมาเพื่อใช้ในกรณีนี้

ข้อดีของ SOAP ก็คือ SOAP ไม่ขึ้นกับคอมพิวเตอร์เทคโนโลยี และภาษาการเขียนโปรแกรมใดๆ สามารถเขียนได้ง่ายและขยายเพิ่มเติมได้

5.2 ส่วนประกอบของ SOAP

SOAP เมสเสจใช้ไวยากรณ์ของ XML ในการสร้าง ประกอบด้วย 3 อีลีเมนต์มาตรฐาน คือ SOAP Envelope, SOAP Header และ SOAP Body



รูปที่ 5-1 โครงสร้างของ SOAP

SOAP เมสเสจ จะต้องเป็นไปตามกฎนี้

- Envelope เป็นอีลีเมนต์ที่อยู่บนสุด ต้องมีอีลีเมนต์นี้เสมอ
- Header อาจจะมีหรือไม่มีก็ได้ แต่ถ้ามีต้องเป็น child element แรกของ Envelope
- Body ต้องเป็น child element แรกของ envelope หรือ header

ตัวอย่างของ SOAP เมสเสจ ตัวอย่างนี้คือ GetLastTradePrice SOAP Request ที่ส่งไปยัง StockQuote service ประกอบด้วยสตริงพารามิเตอร์ symbol และคืนค่า float กลับมากับ SOAP response SOAP envelope element อยู่ที่จุดบนสุดของเอกสาร XML XML namespace ใช้เพื่อป้องกันการสับสนของ SOAP identifier

SOAP request จะเป็นดังนี้

```
POST /StockQuote HTTP/1.1
Host: www.stockquoteserver.com
Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn
SOAPAction: "Some-URI"

<SOAP-ENV:Envelope
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <SOAP-ENV:Body>
    <m:GetLastTradePrice xmlns:m="Some-URI">
      <symbol>DIS</symbol>
    </m:GetLastTradePrice>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP response จะเป็นดังนี้

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn

<SOAP-ENV:Envelope
```

```

xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding"/>
<SOAP-ENV:Body>
  <m:GetLastTradePriceResponse xmlns:m="Some-URI">
    <Price>34.5</Price>
  </m:GetLastTradePriceResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

นอกจากนั้นใน SOAP เมสเสจ จะมีการใช้ XML namespace ทุกๆ อีลีเมนต์ในเอกสารจะขึ้นต้นด้วย namespace namespace จะถูกกำหนดโดยใช้ xmlns attribute ดังนี้

```

<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">

```

แต่หากไม่ต้องการให้ขึ้นต้นด้วย namespace ก็สามารรถทำได้ ดังนี้

```

<Envelope
  xmlns="http://schemas.xmlsoap.org/soap/envelope/">

```

5.2.1 Envelope

envelope element เป็นอีลีเมนต์บับสุดท้ายของเอกสาร XML ที่ใช้แสดงเมสเสจ อาจประกอบด้วยแอตทริบิวต์ คือ envelope namespace และ encodingStyle

1. Envelope Namespace – SOAP เมสเสจจะแสดงเวอร์ชันโดยใช้ namespace ของ envelope element namespace จะถูกนำไปใช้ ขึ้นต้น envelope, header และ body element
2. EncodingStyle Attribute – ใช้แสดงวิธี serialization ของ SOAP เมสเสจ แอตทริบิวต์นี้สามารถ

มีได้ในทุกอีลีเมนต์และจะมีผลกับ content ของอีลีเมนต์นั้นและ child element ทั้งหมดของมันที่ไม่ได้ประกาศแอตทริบิวต์

5.2.2 Body

เป็นส่วนของข้อมูลที่ใช้ในการแลกเปลี่ยน โดยทั่ว ๆ ไปข้อมูลจะเป็นการ marshal RPC call และ error report child element ของ body element จะถูกเรียกว่า Body entry body entry จะต้องเป็นไปตามกฎนี้

- body entry จะต้องแสดงด้วยชื่อเต็มประกอบด้วย namespace URI และ ชื่อของมัน (local name)
- SOAP encodingStyle attribute อาจจะมีได้ เพื่อแสดงถึงวิธี encode ของ body entry นั้น

5.2.3 Header

เป็นส่วนเพิ่มเติมของเมสเสจสามารถประกอบด้วยอินฟอร์เมชันที่ระบุไปยังแอปพลิเคชัน ส่วนเพิ่มเติมนี้อาจนำไปใช้อิมพลิเมนต์เป็น authentication, transaction management เป็นต้น ทุก ๆ child element ของ header element จะถูกเรียกว่า Header entry header entry จะต้องเป็นไปตามกฎดังนี้

- header entry จะต้องแสดงด้วยชื่อเต็มประกอบด้วย namespace URI และ ชื่อของมัน (local name)
- อาจมี SOAP encodingStyle attribute ที่ใช้สำหรับ header entry
- SOAP mustUnderstand attribute และ SOAP actor attribute จะมีหรือไม่ก็ได้ เพื่อใช้บอกว่าจะจัดการกับ entry นั้นอย่างไรและโดยใคร

```
<soap:Header>
<m:local xmlns:m="http://www.w3schools.com/local/"
soap:actor="http://www.w3schools.com/appml" />
<m:language>en</m:language>
<m:currency>USD</m:currency>
</m:local>
</soap:Header>
```

1) SOAP Actor Attribute – SOAP เมสเสจสามารถถูกส่งไปยังปลายทางผ่านตัวกลางของ SOAP (SOAP intermediary) ตัวกลางของ SOAP คือแอปพลิเคชันที่มีความสามารถทั้งรับและส่งต่อ SOAP เมสเสจ ในการส่งต่อตัวกลางจะต้องไม่ส่งต่อ header element ไปให้กับแอปพลิเคชัน โดยตัวกลางนี้จะถูกกำหนดเป็น URI ถ้าไม่มี attribute นี้หมายถึงผู้รับ SOAP เมสเสจเป็นปลายทางสุดท้าย

2) SOAP MustUnderstand Attribute – ใช้แสดงว่า header entry นี้จำเป็นหรือเป็นเพียงออปชันสำหรับผู้รับที่จะนำไปประมวลผล ค่าของมันคือ “1” หรือ “0” ถ้าไม่มีการระบุจะมีค่าเท่ากับ “0” โดย “0” หมายถึงเป็นออปชัน และ “1” หมายถึงจำเป็น โดยจะต้องประมวลผลให้ถูกต้องตาม semantic หรือต้อง fail เช่น เมสเสจเป็นส่วนหนึ่งของ transaction ถ้าปลายทางไม่รองรับ transaction จะต้องไม่ประมวลผลและส่งผลผิดพลาดกลับไปในกรณี ที่ mustUnderstand attribute เป็น “1” แต่ถ้าเป็น “0” จะยังคงประมวลผลต่อไป

5.3 SOAP Fault Element

ข้อความแสดงความผิดพลาดจากแอปพลิเคชันของ SOAP จะเก็บอยู่ใน fault element ซึ่งถ้ามีจะตั้
องปรากฏใน body element เพียงครั้งเดียวใน SOAP เมสเสจ ตัวอย่างอาจเป นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<soap:Body>
<soap:Fault>
<faultcode>soap:MustUnderstand</faultcode>
<faultstring>Mandatory Header error.</faultstring>
<faultactor>http://www.wrox.com/heroes/endpoint.asp</faultactor>
<detail>
<w:source xmlns:w="http://www.wrox.com/">
<module>endpoint.asp</module>
<line>203</line>
</w:source>
</detail>
</soap:Fault>
</soap:Body>
</soap:Envelope>
```

SOAP fault element มี element ย่อย ๆ ดังตารางนี้

Sub Element	Description
<faultcode>	โค้ดที่ระบุถึงการ error
<faultstring>	ข้อความการ error
<faultactor>	ใครเป็นสาเหตุของการ error
<detail>	ระบุอินฟอร์เมชันของการ error

ตารางที่ 5-1 SOAP Fault Element

ค่าของ faultcode สามารถมีค่าได้ ดังตารางนี้

Error	Description
VersionMismatch	namespace ภายใน SOAP Envelope element ไม่ถูกต้อง
MustUnderstand	child element ของ Header element กับ mustUnderstand attribute ที่มีค่า "1" ผู้รับไม่รองรับ
Client	เมสเซจมีรูปแบบไม่ถูกต้อง หรือมีอินฟอร์เมชันที่ไม่ถูกต้อง
Server	เกิดปัญหากับเซิร์ฟเวอร์ ไม่สามารถประมวลผลได้

ตารางที่ 5-2 ค่า faultcode

5.4 SOAP Encoding

SOAP encoding มีวิธีการ map จาก type ของโปรแกรมมิ่งไปเป็น XML 2 วิธี คือจากภายนอก โดยใช้ WSDL (Web Services Description Language) ที่บอกถึง type ของข้อมูลที่รับหรือส่ง หรือใช้ xsi:typeattribute ในกรณีที่ใช้ภาษาที่ไม่รองรับ WSDL โดยทั้ง 2 วิธีจะใช้ XML schema ในการระบุ type ตัวอย่าง ของการใช้ xsi จะเป็นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>
    <m:MixedMessage xmlns:m="http://www.wrox.com/mix/">
      <param1 xsi:type="xsd:string">OU812</param1>
      <param2 xsi:type="xsd:integer">2001</param2>
      <param3 xsi:type="xsd:double">3.14159</param3>
    </m:MixedMessage>
  </soap:Body>
</soap:Envelope>
```

SOAP รองรับ type ของข้อมูล ได้ทั้ง simple type และ compound type เช่น struct และ array

5.4.1 ตัวอย่าง Struct

struct book ที่เขียนด้วยภาษา c++ จะเป็นดังนี้

```
struct Book
{
    string author;
    string name;
    int page;
};
```

Soap เมสเซจที่ไม่ใช้ WSDL แต่ใช้ xsi จะเป็นดังนี้

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:searchBook xmlns:ns1="http://soapinterop.org/" SOAPENV:
      encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <bookResult xmlns:ns2="http://soapinterop.org/xsd" xsi:type="ns2:Book">
        <author xsi:type="xsd:string">Ed Roman</author>
        <name xsi:type="xsd:string">Mastering EJB</intro>
        <page xsi:type="xsd:int">500</page>
      </bookResult>
    </ns1:searchBook>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

แต่ถ้าใช้ WSDL ในส่วนของการประกาศ schema จะต้องประกอบด้วย schema ดังนี้

```
<types>
<schema targetNamespace='http://soapinterop.org/xsd'
xmlns='http://www.w3.org/2001/XMLSchema'
xmlns:SOAP-ENC='http://schemas.xmlsoap.org/soap/encoding/'
xmlns:wSDL='http://schemas.xmlsoap.org/wSDL/'
```

```

elementFormDefault='qualified'>
<complexType name="Book">
<sequence>
<element name='author' type='string'/>
<element name='name' type='string'/>
<element name='page' type='int'/>
</sequence>
</complexType>
</schema>
</types>

```

5.4.2 ตัวอย่าง Array

ตัวอย่างนี้เป็น array ของ struct book ในตัวอย่างก่อน Soap เมสเสจที่ไม่ใช่ WSDL แต่ใช้ xsi จะเป็นดังนี้

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/1999/XMLSchema">
<SOAP-ENV:Body>
<ns1:BookArrayResult xmlns:ns1="http://soapinterop.org/" SOAPENV:
encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<BookArray xmlns:ns2="http://schemas.xmlsoap.org/soap/encoding/"
xsi:type="ns2:Array" xmlns:ns3="http://soapinterop.org/xsd"
ns2:arrayType="ns3:Book[3]">
<item xsi:type="ns3:Book">
<author xsi:type="xsd:string">Ed Roman</author>
<name xsi:type="xsd:string">Mastering EJB</name>
<page xsi:type="xsd:int">500</page>
</item>
<item xsi:type="ns3:Book">
<author xsi:type="xsd:string">Roger Session</author>
<name xsi:type="xsd:string">The Battle of the middleware</name>
<page xsi:type="xsd:int">350</page>

```

```

</item>
<item xsi:type="ns3:Book">
<author xsi:type="xsd:string">Chris Dix</author>
<name xsi:type="xsd:string">Professional XML Web Service</name>
<page xsi:type="xsd:int">550</page>
</item>
</BookArray>
</ns1:BookArrayResult>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

แต่ถ้าใช้ WSDL ในส่วนของการประกาศ schema จะต้องประกอบด้วย schema ดังนี้

```

<types>
<schema targetNamespace='http://soapinterop.org/xsd'
xmlns='http://www.w3.org/2001/XMLSchema'
xmlns:SOAP-ENC='http://schemas.xmlsoap.org/soap/encoding/'

xmlns:wsdl='http://schemas.xmlsoap.org/wsdl/'
elementFormDefault='qualified'>
<complexType name="Book">
<sequence>
<element name='author' type='string'/>
<element name='name' type='string'/>
<element name='page' type='int'/>
</sequence>
</complexType>
<complexType name='ArrayOfBook'>
<complexContent>
<restriction base='SOAP-ENC:Array'>
<attribute ref='SOAP-ENC:arrayType' wsdl:arrayType='typens:Book[]'/>
</restriction>
</complexContent>
</complexType>

```

```
</schema>
</types>
```

5.5 SOAP ใน HTTP

ในการส่ง SOAP ผ่านทาง HTTP จะต้องใช้ content-type เป็น text/xml แต่ใน SOAP request จะต้องมี header SOAPAction ภายใน HTTP header SOAPAction จะเป็นตัวบอกให้เซิร์ฟเวอร์รู้ว่า HTTP Post นั้นเป็น SOAP เมสเสจ และค่าของ header คือ URI ที่แสดงถึงจุดหมายของ SOAP เมสเสจ ส่วน SOAP response จะต้องมีการ status code ตามมาตรฐานของ HTTP โดย 200-299 แสดงว่าสำเร็จ แต่ถ้า response เมสเสจ เป็นการ fault แล้ว status code จะต้องเป็น 500 ซึ่งแสดงถึง internal server error ตัวอย่างของ response ที่มี status code เป็น 500 อาจเป็นดังนี้

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml
Content-Length: ###
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/1999/XMLSchema">
<soap:Body>
<soap:Fault>
<faultcode>soap:VersionMismatch</faultcode>
<faultstring>The SOAP namespace is incorrect.</faultstring>
<faultactor>http://www.wrox.com/endpoint.asp</faultactor>
<detail>
<w:errorinfo xmlns:w="http://www.wrox.com/">
<desc>The SOAP namespace was blank.</desc>
</w:errorinfo>
</detail>
</soap:Fault>
</soap:Body>
</soap:Envelope>
```

5.6 SOAP สำหรับ RPC

จุดประสงค์ของการออกแบบ SOAP คือทำ RPC โดยใช้ XML การเรียก RPC นั้นจะ map เข้ากับ HTTP request และ RPC response จะ map กับ HTTP response

ในการทำ method call จำเป็นที่จะต้องมียีนฟอร์เมชันดังนี้

1. URI ของออบเจ็กต์ปลายทาง (target object)
 2. ชื่อของเมธอด
 3. คำอธิบายของเมธอด (method signature) ส่วนนี้เป็นออปชัน
 4. พารามิเตอร์สำหรับเมธอด
 5. ข้อมูลส่วนหัว (header data) เป็นออปชัน
- พิจารณาเมธอดดังนี้

```
double GetStockQuote ( [in] string sSymbol);
```

ถ้า namespace ของเมธอดคือ “http://www.worxstox.com/” แล้วการเรียกเมธอดโดย request stock quote ใช้ symbol OU812 จะเป็นดังนี้

```
<q:GetStockQuote xmlns:q="http://www.worxstox.com/">
<q:sSymbol xsi:type="xsd:string">OU812</q:sSymbol>
</q:GetStockQuote>
```

ชื่อเมธอดและชื่อของอีลีเมนต์จะต้อง match กันเช่นเดียวกับพารามิเตอร์
สำหรับ response จะต้องชื่ออีลีเมนต์ต้องเป็นชื่อเมธอดตามด้วย response ดังนี้

```
<q:GetStockQuoteResponse xmlns:q="http://www.worxstox.com/">
<q:ret xsi:type="xsd:double">100.0</q:ret>
</q:GetStockQuoteResponse>
```

ชื่ออีลีเมนต์ของค่าที่คืนมาสามารถเปลี่ยนได้ โดยจะกำหนดได้จากโปรแกรมที่เขียนหรือจาก WSDL

5.7 SOAP Toolkit

SOAP toolkit คือเครื่องมือที่จะทำหน้าที่ในการประมวลผล SOAP request และส่ง response กลับไปให้ไคลเอนต์ โดย SOAP toolkit แต่ละตัวใช้ได้บนแพลตฟอร์มที่ต่างกัน รองรับเทคโนโลยี web service ต่างกัน บางตัวอาจจะรองรับ WSDL หรือ UDDI ในขณะที่บางตัวรองรับอย่างใดอย่างหนึ่งหรือไม่รองรับทั้งสองอย่าง และถึงแม้ว่า SOAP จะเป็นอิสระต่อแพลตฟอร์ม แต่ใน SOAP toolkit บางตัวยังไม่สามารถทำงานข้ามผลิตภัณฑ์ (interoperability) ได้ เนื่องจากรองรับเทคโนโลยีของ web service ไม่เหมือนกัน เช่น xsi ในบางผลิตภัณฑ์จะบังคับให้ SOAP เมสเสจจะต้องระบุ type ด้วย xsi หากไม่มีก็จะ failur ซึ่งในปัจจุบันแต่ละผลิตภัณฑ์ก็พยายามพัฒนาให้สามารถทำงานด้วยกันได้

ชื่อ	แพลตฟอร์ม	แหล่งที่มา
4s4c	COM	http://www.4s4c.com
A SOAP for RPC NT Service	COM	http://www.whitemesa.com
Apache Axis	Java	http://xml.apache.org/axis
Apache SOAP	Java	http://xml.apache.org/soap
CapeConnect	Java	http://www.capeclear.com
Glue	Java	http://www.themindelectric.com
IBM Web Services Toolkit	Java	http://www.alphaworks.ibm.com
IdooXoop for Java and C++	Java , C++	http://www.zvon.org
Microsoft SOAP Toolkit	COM	http://www.microsoft.com
PocketSOAP	COM	http://www.pocketsoap.com
SOAP::Lite	Perl	http://www.soaplite.com
Visual Studio .NET	COM	http://www.microsoft.com

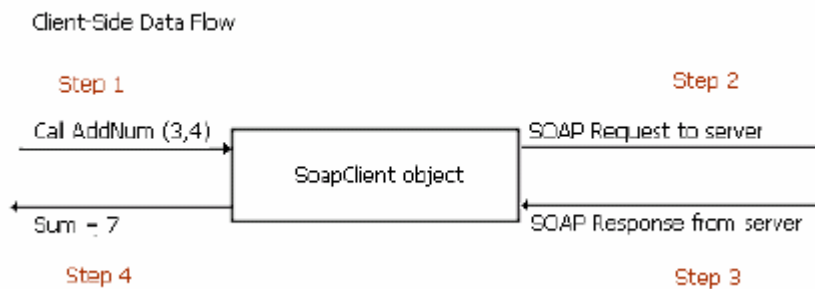
ตารางที่ 5-3 SOAP Toolkit

5.8 โครงสร้างภายในของ SOAP Toolkit ของ Microsoft

The Web Services Description Language (WSDL) เป็นรูปแบบหนึ่งของ XML เพื่อใช้อธิบายว่าเซิร์ฟเวอร์นั้นมีเซอร์วิส (Services) และ operation ภายในเซอร์วิสอะไรให้ใช้บ้าง ซึ่ง operation ภายในเซอร์วิสนั้นจะอธิบายรูปแบบที่ไคลเอนต์ (client) ต้องใช้ในการเรียก operation The Web Services Meta Language (WSML) file เป็นส่วนที่เพิ่มขึ้นมาจากไฟล์ WSDL เพื่อใช้ในการเชื่อมโยงแต่ละ operation ในเซอร์วิสกับ COM object โดย WSML file จะพิจารณาว่า COM object ตัวใดที่จะต้องถูกโหลด (load) ขึ้นมาเมื่อมีการเรียกใช้ operation ในเซอร์วิสทั้งไฟล์ WSDL และ WSML จะต้องอยู่ที่เซิร์ฟเวอร์ โดยไคลเอนต์ที่ต้องการที่จะส่ง SOAP request ไปที่เซิร์ฟเวอร์ จะต้องได้รับ WSDL จากเซิร์ฟเวอร์ก่อน เพื่อหารูรูปแบบของ SOAP request ในการติดต่อ

5.8.1 การสร้าง SoapClient Object

ทางฝั่งไคลเอนต์ เมื่อแอปพลิเคชันส่ง message ไปที่ Soap Client object เพื่อเรียกใช้ operation หนึ่งๆนั้น(ตามรูปนี้คือการบวกเลข 2 ตัว) SoapClient object จะทำการสร้าง SOAP request ส่งไปที่เซิร์ฟเวอร์ เมื่อเซิร์ฟเวอร์ทำการประมวลผลเสร็จเรียบร้อยแล้ว ก็จะส่งผลลัพธ์ในรูปแบบของ SOAP response กลับมาให้ไคลเอนต์ หลังจากนั้น SoapClient object จจะทำการแปลและส่ง message ที่เก็บผลลัพธ์ของ operation นั้นกลับไปให้แอปพลิเคชัน ตามไดอะแกรมด้านล่าง



รูปที่ 5-2 แสดงการ Client-Side Data Flow

แอปพลิเคชันต้อง instantiate Soap object ก่อนที่จะทำการเรียกใช้ operation ใดๆ (SoapClient object นี้ถูกสร้างเก็บไว้ในไฟล์ MSSOAP1.dll) โดยใช้เมธอด SoapClient.mssoapinit (ชื่อไฟล์ WSDL, ชื่อเซอร์วิส, ข้อมูลพอร์ต) โดยข้อมูลพอร์ตจะใช้ในการแบ่งแยกช่องทางที่ใช้ในการส่ง SOAP message ในระหว่าง initialize SoapClient object จะตรวจสอบเมธอดทั้งหมดที่อยู่ในไฟล์ WSDL ด้วยการทำงานแบบไดนามิกนี้ทำให้เราสามารถที่จะเรียกใช้เมธอดใดก็ได้ที่ได้อธิบายไว้ในไฟล์ WSDL ดังแสดงตาม Microsoft Visual Basic Scripting Edition (VBScript) code ด้านล่างนี้

```

SET soapclient = CreateObject("MSSOAP.SoapClient")

Call soapclient.mssoapinit("http://localhost/MyWSDL.wsdl", "TestService", "TestPort")

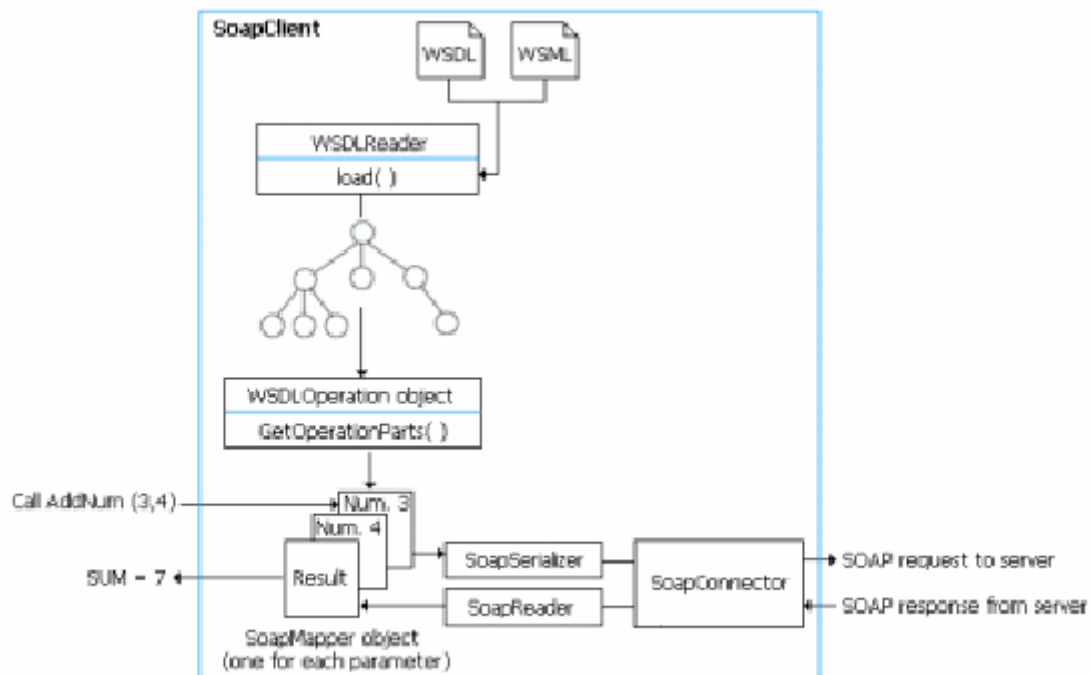
' Now the client can call an operation listed in the portType element
' specified when calling mssoapinit().

wscript.echo soapclient.AddNumbers(2, 3)

```

5.8.2 การประมวลผลภายใน SoapClient Object

การทำงานภายใน SoapClient object แสดงได้ดังรูปด้านล่าง



รูปที่ 5-3 แสดงโครงสร้างภายในของ SOAP

WSDL Reader object จะทำการโหลดไฟล์ WSDL และไฟล์ WSML เข้าไปใน DOM แล้วทำการวิเคราะห์ โดย WSDLReader object จะสร้าง WSDLOperation object เพื่อทำการเรียกใช้งาน operation (ตามรูปนี้คือ AddNum(3, 4)) หลังจากนั้น WSDLOperation จะเรียกเมธอด GetOperationParts เพื่อดึงรูปแบบ message ของอินพุตและเอาต์พุตของ operation ออกมา แล้ว SoapClient จะสร้าง Soap Mapper objects ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้มาจากการเรียกใช้ operation เข้าไว้ใน objects เหล่านี้ พอถึงขั้นตอนนี้ SoapSerializer objects จะสร้าง SOAP request message จาก SoapMapper objects ที่เหมาะสมและส่งไปที่เซิร์ฟเวอร์เมื่อเซิร์ฟเวอร์ประมวลผลเสร็จแล้วก็จะส่ง SOAP response กลับไปที่ไคลเอนต์ SoapClient object จะทำการโหลดผลลัพธ์ของ operation ไปเก็บใน SoapMapper object ที่เหมาะสมแล้วส่งผลลัพธ์กลับไปให้แอปพลิเคชันของผู้ใช้

5.8.3 การสร้าง SoapServer Object

ส่วนทางฝั่งเซิร์ฟเวอร์ เราสามารถกำหนดรูปแบบการรับ SOAP request (SOAP listener) ได้ 2 รูปแบบ คือ Internet Server API (ISAPI) server และ Active Server Pages (ASP) server ในไฟล์ WSDL จะมี URL ที่ระบุว่าจะใช้ไฟล์ใดบนเซิร์ฟเวอร์ในการรับ SOAP request เพื่อใช้ในการพิจารณาว่าจะรับ SOAP request อย่างไรบนเซิร์ฟเวอร์ ตามโค้ดด้านล่างนี้ ตัวพิมพ์หนาจะเป็นส่วนที่ใช้ในการเรียกไปที่ ISAPI listener

```

<definitions>
...
<service name='DocSample1' >
  <port name='DocSample1PortType' binding='tns:DocSample1Binding' >
    <soap:address
      location='http://localhost/DocSample1Test/DocSample1.wsdl' />
    </port>
  </service>
...
</definitions>

```

การทำงานทางฝั่งเซิร์ฟเวอร์

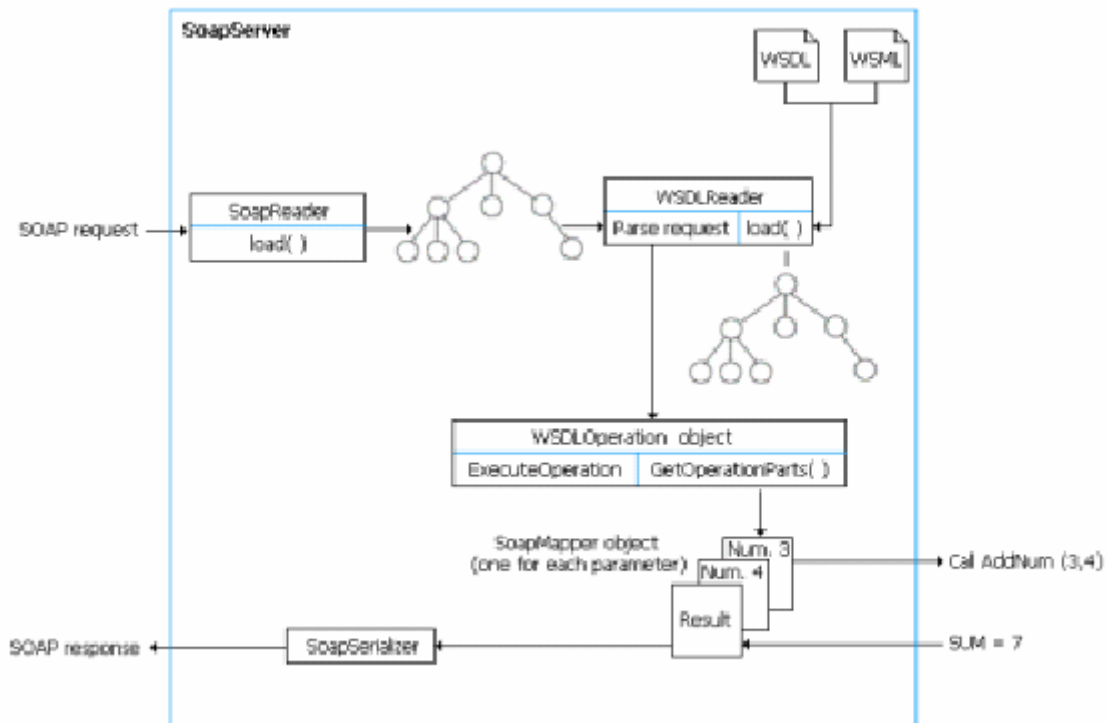
ไม่ว่า SOAP request จะเรียกไปยัง ISAPI หรือ ASP listener ก็ตาม ข้อมูลที่รับส่งก็เหมือนกัน เมื่อ SoapServer object ได้รับ SOAP request จากไคลเอนต์ ก็จะทำการแปลและเรียกไปยังเมธอด COM ที่ตรงกับ operation ที่ต้องการ (ตามไคอะแกรมนี้คือเมธอด AddNum) หลังจากนั้นเมธอดที่ถูกเรียกใช้นั้นก็ จะส่งผลลัพธ์กลับมาให้ SoapServer object ซึ่งมันก็จะแปลงผลลัพธ์ให้อยู่ในรูปของ SOAP response แล้วจึงส่งกลับไปที่ไคลเอนต์



รูปที่ 5-4 แสดง Server-Side Data Flow

5.8.4 การประมวลผลภายใน SoapServer Object

การทำงานภายใน SoapServer object แสดงได้ดังรูปด้านล่าง



รูปที่ 5-5 แสดงโครงสร้างภายใน SOAP Server

เมื่อได้รับ SOAP request จากไคลเอนต์แล้ว SoapReader object ก็จะโหลด request message ไปไว้ใน DOM structure หนึ่ง ในขณะที่ WSDLReader จะโหลดไฟล์ WSDL และไฟล์ WSML ไปไว้ในอีก DOM structure หนึ่ง ตัว WSDLReader จะแปล request และสร้าง WSDLOperation object ขึ้นเพื่อใช้ใน requested operation หลังจากนั้น WSDLOperation object จะเรียกใช้เมธอด GetOperationParts เพื่อดึงรูปแบบ message ของอินพุตและเอาต์พุตของ operation ออกมา แล้ว SoapServer จะสร้าง Soap Mapper objects ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้มาจากการเรียกใช้ operation เข้าไว้ใน objects เหล่านี้ พอถึงขั้นตอนนี้ SoapServer object จะทำการเรียกไปที่ COM method ที่ตรงกับที่ร้องขอเข้ามาเมื่อ COM method ประมวลผลเสร็จแล้วก็จะส่งผลลัพธ์กลับไปให้ SoapServer object ซึ่ง SoapServer object ก็จะทำการโหลดผลลัพธ์ของ operation ไปเก็บใน SoapMapper object ที่เหมาะสม แล้วจึงใช้ SoapSerializer object ส่ง SOAP response message กลับไปให้ไคลเอนต์แอปพลิเคชันของผู้ใช้

5.9 การสร้าง SOAP Client และ SOAP Server ด้วย .NET Framework

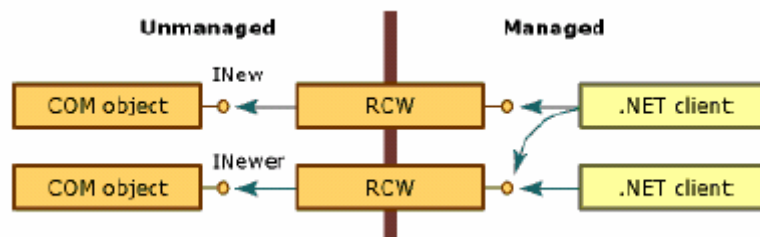
5.9.1 การใช้ ASP.NET ทำ SOAP Web Service

ใช้ความสามารถในการทำ COM+ Interop โดยสร้างคลาสของ ASP.NET มาเรียกใช้งาน businesslogic ที่สร้างด้วย VB6



รูปที่ 5-6 การใช้ ASP.NET ทำ SOAP web service

อาศัยความสามารถในการทำ marshal / unmarshal object ให้อยู่ในรูปแบบ SOAP ของ ASP.NET โดย ASP.NET ต้องทำ Interop โดยอาศัย Runtime Callable Wrapper ช่วยจำลอง COM object ให้ .NET มองเห็นเป็น .NET object ธรรมดา



รูปที่ 5-7 Runtime Callable Wrapper

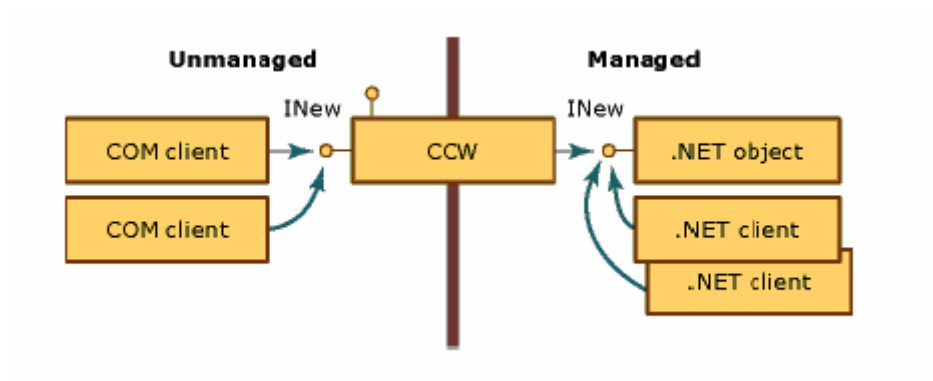
5.9.2 การใช้ .NET ทำ SOAP Client

ในทางกลับกันที่ COM ต้องการใช้บริการจาก SOAP ก็กระทำผ่าน .NET web service client



รูปที่ 5-8 การใช้ .NET ทำ SOAP client

แต่การที่จะทำให้ COM ติดต่อกับ .NET class ได้นั้นต้องผ่านกระบวนการ COM Interop โดยการสร้าง COM callable wrapper ซึ่งจะช่วยให้ COM สามารถเห็น .NET class เป็น COM object ธรรมดา



รูปที่ 5-9 COM callable wrapper ที่ทำให้ COM client สามารถเห็น .NET object